

Lección 4. Transformación de datos normalización codificación y escalado

Por qué es importante transformar los datos antes de entrenar modelos de aprendizaje automático

Cuando se entrena un modelo de aprendizaje automático, es habitual que las características tengan escalas muy diferentes. Por ejemplo, algunas variables pueden medir ingresos en miles o millones, mientras otras registran la edad en años o la puntuación de un examen de 0 a 10. Esta diferencia tan grande provoca que el modelo dé más peso a las variables con valores más altos, aunque no sean necesariamente las más relevantes, lo que puede afectar la calidad y estabilidad del entrenamiento.

Además, muchos algoritmos funcionan mejor si las características tienen distribuciones más uniformes y menos sesgadas. Cuando los datos tienen distribuciones muy asimétricas o incluyen valores extremos, los modelos pueden entrenar más lento, cometer más errores o incluso fallar al predecir datos nuevos.

Por otra parte, muchas variables importantes son categóricas, como colores, niveles educativos o tipos de productos, y los algoritmos no pueden interpretarlas directamente si no están en formato numérico. Por eso es necesario aplicar una transformación adicional llamada codificación, que convierte estos valores en números para que los modelos puedan procesarlos correctamente.

Estas transformaciones permiten que cada variable aporte información de forma justa y equilibrada. Incluyen la normalización, que ajusta los valores para que estén en un mismo rango; la estandarización, que asegura que cada característica tenga una media cercana a cero y una dispersión estándar común; y la codificación de variables categóricas, que permite incorporar información no numérica valiosa.

Transformaciones logarítmicas y de potencia

Cuando las variables numéricas tienen distribuciones muy asimétricas o incluyen valores extremos, también conviene aplicar transformaciones logarítmicas o de potencia. La transformación logarítmica convierte los valores en su logaritmo, lo que acerca los valores extremos al rango principal y logra una distribución más simétrica y una varianza más estable. Las transformaciones de potencia, como la raíz cuadrada o Box-Cox, ayudan a ajustar la distribución y a reducir la influencia de los datos más alejados, encontrando el mejor ajuste posible.

Aplicar estas transformaciones hace que el entrenamiento sea más rápido, las predicciones más precisas y estables, y se reduzca el riesgo de sesgos. Esto garantiza resultados más fiables y modelos que generalizan mejor a datos reales. Además, se pueden automatizar fácilmente mediante herramientas como los pipelines, que permiten reproducir todo el proceso desde la limpieza inicial hasta el entrenamiento final, facilitando mucho el trabajo.

Es importante tener en cuenta que no siempre es necesario aplicar estas transformaciones. Son útiles cuando los datos tienen distribuciones muy sesgadas o valores extremos que afectan al

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



rendimiento, pero si las distribuciones ya son equilibradas, podrían no aportar mejoras y hasta complicar el análisis. Por eso, conviene revisar siempre su impacto de forma práctica y decidir si realmente aportan valor al modelo.

Qué significa normalizar los datos y cómo mejora la comparabilidad entre variables

La normalización es un proceso que ajusta las escalas de las variables y las hace comparables entre sí. Sirve para que, aunque las variables midan cosas distintas o tengan unidades diferentes, podamos analizarlas y compararlas sin que unas influyan más que otras solo por tener valores más grandes. Así, todas las variables aportan información de forma justa y se evita que la magnitud de los números determine su importancia.

Existen varias formas de normalización. Una es la normalización Min-Max, que adapta los valores para que estén dentro de un rango fijo, como de cero a uno. Esto se usa cuando queremos que todas las variables tengan la misma escala, por ejemplo al comparar la altura y el peso de personas, que tienen medidas distintas y que, al llevarlas a la misma escala, facilita el análisis.

También está la estandarización o Z-score, que aunque a veces se considera un tipo de normalización, en realidad va un paso más allá. Ajusta los datos para que tengan una media de cero y una dispersión uniforme, mostrando cuánto se alejan o se acercan a la media. Esto es útil cuando queremos comparar variables que tienen distribuciones distintas y ver su relación con respecto a esa media.

Además, existe el escalado robusto, que se usa cuando hay valores muy altos o muy bajos que distorsionan los datos. Ajusta los valores usando la mediana y los rangos centrales, evitando que esos valores extremos tengan tanto peso y distorsionen el análisis.

Por ejemplo, en un conjunto de datos con información sobre la altura de personas y sus salarios, la altura puede ir de 150 a 200 centímetros y el salario de 0 a 5000 euros. Sin normalizar, el salario pesaría mucho más en cualquier comparación, aunque no sea lo más importante. Al normalizar, la altura y el salario se comparan de forma equilibrada y se puede ver mejor cómo influyen en el análisis.

Qué implica estandarizar y escalar los datos y cómo ayuda a mejorar la eficiencia y estabilidad de los modelos

Antes vimos que transformar los datos antes de entrenar modelos de aprendizaje automático es clave para que las variables tengan la misma importancia y el modelo pueda aprender patrones de forma más equilibrada. También hablamos de las transformaciones logarítmicas y de potencia, que corrigen distribuciones asimétricas y reducen la influencia de valores extremos. Y vimos cómo la normalización ajusta las escalas de las variables para que estén en el mismo rango y así mejorar la comparabilidad entre ellas.

Ahora, estandarizar y escalar los datos es otro paso esencial. Estas técnicas también ajustan los valores de las variables, pero lo hacen para que no solo tengan la misma escala, sino también la

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



misma media y una dispersión uniforme. La normalización adapta los datos a un rango fijo, como de cero a uno, mientras que la estandarización transforma los datos para que su media sea cero y su dispersión estándar sea constante. Escalar, en general, es un término que incluye tanto la normalización como la estandarización y otras formas de ajustar las proporciones de los datos para que ninguna variable pese más solo por tener números más grandes.

Por ejemplo, imagina que tienes datos sobre la duración de llamadas telefónicas, que va de 1 a 20 minutos, y el coste mensual de la factura, que va de 10 a 500 euros. Con la normalización, cada variable se ajusta para que vaya de 0 a 1, haciendo que ambas estén en el mismo rango sin importar que antes tuvieran escalas muy distintas. Con la estandarización, los valores se transforman para que tengan una media de cero y muestren cuánto se alejan de esa media, lo que permite compararlos aunque sus distribuciones sean distintas. Así, las dos variables aportan de forma justa y equilibrada al análisis, sin que ninguna domine solo por tener cifras más grandes.

Estas transformaciones son esenciales para que modelos como redes neuronales, clustering o regresiones trabajen mejor. Cuando las variables tienen magnitudes muy distintas, el modelo puede confundir la magnitud con la importancia real, provocando resultados menos precisos, menos estables y entrenamientos más lentos. Estandarizar y escalar permite que el entrenamiento sea más rápido, las predicciones más fiables y que los patrones reales se detecten mejor. La normalización suele ser más útil cuando las variables ya tienen distribuciones similares y solo varían en la escala, mientras que la estandarización resulta más eficaz cuando las variables tienen distribuciones diferentes o cuando los algoritmos necesitan medias y varianzas equilibradas. Por eso, conviene analizar siempre las características de los datos y el tipo de modelo antes de decidir cuál de estas técnicas aporta más valor en cada caso.

Por qué es necesario codificar las variables categóricas para que los algoritmos puedan usarlas correctamente

Muchos modelos de aprendizaje automático solo pueden trabajar con datos numéricos, pero en la práctica hay muchas variables que no lo son, como colores, niveles educativos o tipos de productos. Estas variables categóricas aportan información muy importante, pero si no las transformamos, los algoritmos no pueden interpretarlas ni extraer patrones reales.

Para que estas variables puedan ser procesadas, se aplican técnicas de codificación que las convierten en números. Una de las más usadas es el LabelEncoder, que asigna un número único a cada categoría. Por ejemplo, si tenemos la variable “color” con valores rojo, azul y verde, LabelEncoder podría transformarlos en 0, 1 y 2. Es un método sencillo que se usa cuando la variable no tiene un orden específico o cuando los modelos no dependen de relaciones de magnitud, como en árboles de decisión.

Otra técnica muy común es el OneHotEncoder, que crea una columna para cada categoría y marca con 1 la categoría correspondiente y con 0 las demás. Siguiendo con el ejemplo del color, se crearían tres columnas: “rojo”, “azul” y “verde”. Si el valor original es “azul”, solo la columna “azul” tendrá un 1 y las otras dos un 0. Este método es muy útil para modelos que no pueden interpretar relaciones de orden y para variables con pocas categorías, como cuando se codifican ciudades o tipos de productos.

En algunas situaciones, especialmente cuando las categorías tienen una relación ordinal (como niveles educativos: básico, medio, avanzado), se pueden aplicar métodos más específicos que respeten ese orden para que los modelos lo tengan en cuenta. Además, existen otras técnicas más avanzadas como el Box-Cox, que aunque suele usarse para transformar distribuciones numéricas muy sesgadas, también puede ayudar cuando queremos que las variables numéricas se comporten más como una distribución normal. Por ejemplo, si tenemos una variable “ingresos” con valores muy elevados y pocos valores bajos, Box-Cox transforma esos datos para que los valores extremos estén más equilibrados con el resto, facilitando que el modelo los use de forma más justa.

Estas no son las únicas técnicas, ya que hay métodos adicionales como el OrdinalEncoder o las codificaciones basadas en frecuencia o en conteo, que pueden ayudar cuando tenemos muchas categorías o cuando queremos que el modelo entienda mejor las relaciones entre ellas.

Elegir la codificación correcta depende del tipo de variable y del modelo que vayamos a utilizar. LabelEncoder se usa cuando la variable es nominal y el modelo no necesita entender distancias entre categorías. OneHotEncoder es más eficaz cuando las categorías no tienen orden y queremos evitar que el modelo suponga relaciones numéricas que no existen. Box-Cox o transformaciones similares se aplican cuando necesitamos corregir distribuciones muy asimétricas en datos numéricos o continuos.

En resumen, codificar las variables categóricas es esencial para que los modelos puedan usar la información que aportan y convertirla en patrones útiles y comparables con el resto de los datos. Elegir el tipo de codificación adecuado mejora la calidad de los datos y la precisión de las predicciones.

Cómo usar pipelines para encadenar limpieza codificación y escalado de forma reproducible

Después de ver cómo la normalización, la estandarización y la codificación de variables categóricas ayudan a que los datos sean comparables y relevantes para los modelos, es importante que todos estos pasos se apliquen siempre igual y en el orden correcto. Aquí es donde entran los pipelines, que permiten encadenar todo este proceso de transformación de datos en un único flujo claro y reproducible. De esta forma, el modelo recibe siempre los datos procesados de la misma manera, sin errores o diferencias inesperadas.

Un pipeline puede incluir desde la limpieza inicial de los datos, eliminando valores nulos o corrigiendo errores, hasta la eliminación de campos que no aportan valor al análisis, como los identificadores. Además, puede incorporar pasos para detectar y ajustar outliers o valores extremos, que podrían distorsionar los cálculos si no se tratan adecuadamente. Después, añade la codificación de variables categóricas, usando herramientas como LabelEncoder para variables con un único valor numérico, OneHotEncoder para crear columnas separadas por categoría o Box-Cox para corregir distribuciones de variables numéricas con valores muy sesgados.

Una vez las variables categóricas están convertidas en formato numérico, el pipeline también puede aplicar la normalización o la estandarización. La normalización ajusta los datos a un rango fijo, como de cero a uno, para que todas las variables tengan la misma escala. La estandarización, en cambio, los transforma para que tengan media cero y varianza constante, ayudando a comparar mejor

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



variables con distribuciones distintas. El escalado, en general, engloba todas estas técnicas que igualan las proporciones y eliminan la influencia de magnitudes artificiales.

Por ejemplo, si tienes un conjunto de datos con la “edad” (numérica), la “ciudad” (categórica), los “ingresos” (numéricos, con valores extremos) y un identificador de cliente, el pipeline podría empezar eliminando el identificador, rellenar los valores nulos de la “edad”, detectar y ajustar los outliers de “ingresos”, aplicar OneHotEncoder a la “ciudad”, corregir la distribución de “ingresos” con Box-Cox y finalmente estandarizar todas las variables numéricas. Con este flujo ordenado y automático, se evitan errores y se garantiza que el modelo reciba siempre los datos en condiciones óptimas para aprender.

En la práctica, los pipelines permiten reproducir todo este flujo fácilmente, facilitando que el análisis sea consistente y fiable, sin necesidad de rehacer los pasos manualmente. Además, ayudan a que los modelos entrenen más rápido y obtengan predicciones más precisas y estables.

En resumen, los pipelines permiten encadenar la limpieza de datos, la eliminación de campos innecesarios, el tratamiento de outliers, la codificación de variables categóricas y el escalado de forma segura y organizada. Son clave para que los modelos aprendan patrones reales y para que los datos se preparen siempre igual, garantizando resultados más consistentes y confiables.

Cómo aplicar la secuencia de transformación de datos

Transformar los datos antes de entrenar modelos es fundamental para que las variables aporten información de forma equilibrada y no generen sesgos artificiales. Aunque existe un orden general en el que se suelen aplicar estos pasos, la secuencia exacta siempre depende de las características concretas de los datos y del tipo de modelo que se use.

Por lo general, primero se limpia la información eliminando errores, valores nulos y campos que no aportan valor. Después se suele revisar la presencia de outliers y decidir si conviene ajustarlos o no, teniendo en cuenta que a veces son valores reales que no deben descartarse. La codificación de variables categóricas suele hacerse a continuación para que los modelos puedan procesarlas como números. Una vez están todas las variables numéricas, se pueden aplicar transformaciones para corregir distribuciones muy sesgadas, como el logaritmo o Box-Cox.

El paso final es la normalización o la estandarización, que ayuda a que todas las variables estén en escalas comparables o que tengan medias y varianzas similares. Sin embargo, no todos estos pasos son imprescindibles en todos los proyectos. Depende mucho de cómo sean las distribuciones de los datos y de las exigencias de cada modelo.

Por eso es clave revisar visualmente las distribuciones y analizar la influencia real de cada paso antes de decidir si aplicarlo o no. Usar herramientas como los pipelines facilita todo este proceso, porque permiten encadenar cada etapa de forma ordenada y reproducible.